# OBJECT ORIENTED PROGRAMMING (OOP)

---

## Project Report: JavaFX Minesweeper

---


## Group Members:

- Ahmad Ali Khan : SP25 - BSE - 010
- Athar Abbas : SP25 - BSE - 082
- Moaz Nadeem : SP25 - BSE - 091
- Mukaram Nadeem : SP25 - BSE - 106
- Naeem Akbar : SP25 - BSE- 094


---

## Introduction:

This game is a recreation of Microsoft's classic Minesweeper. Minesweeper was released in 1990 as part of the Windows Entertainment Pack. Curt Johnson and Robet Donner were co-authors of this game.

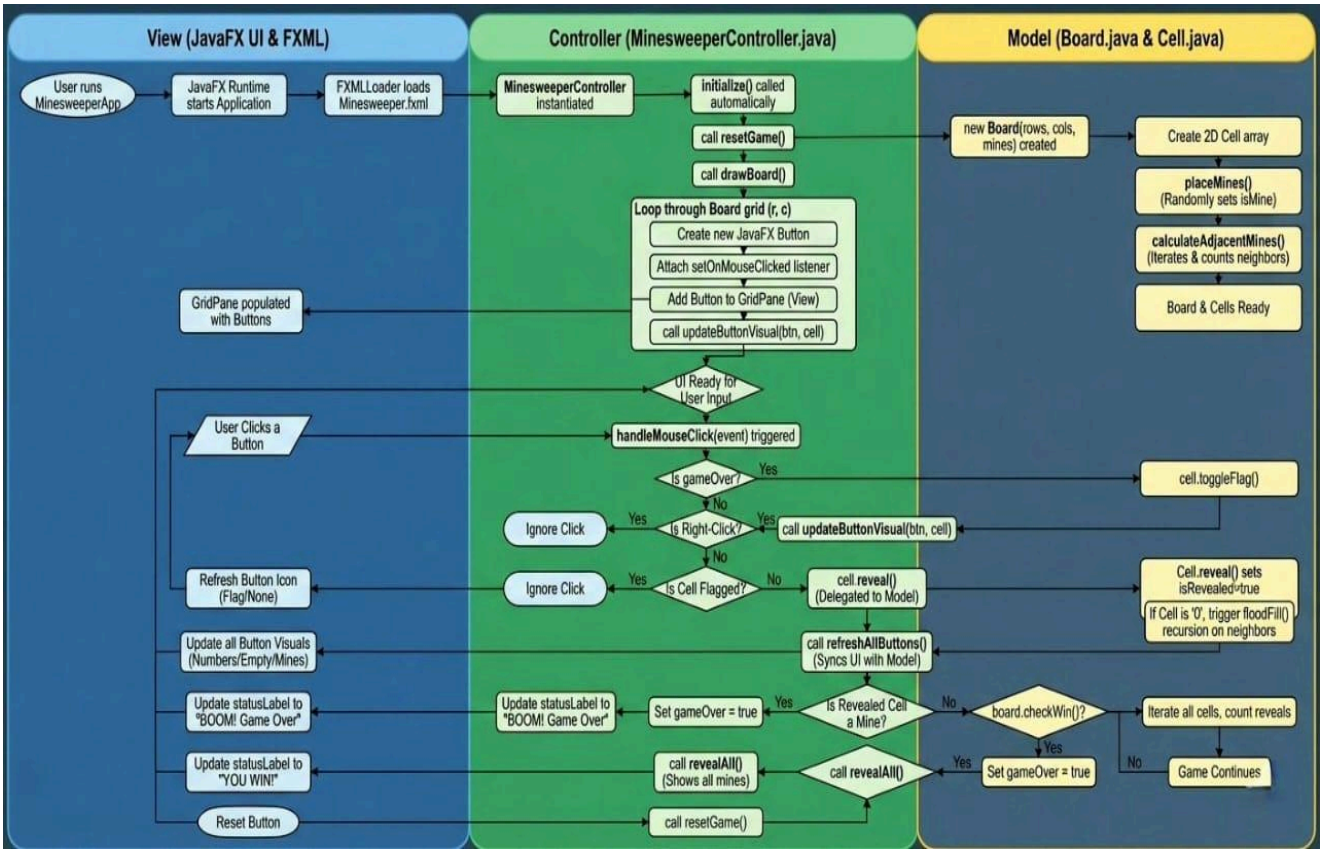
This application is built using Java and the JavaFX framework. The core functionality involves a grid of cells where the player must deduce the location of hidden mines without detonating them, using numerical clues provided by adjacent cells.

## Objectives:

The primary objectives of this project are:

- **Modular Programming:** To demonstrate the separation of concerns by dividing the application into distinct classes such as `Game`, `Board`, `Cell`, and `Minesweeper`.
- **Game Logic Implementation:** To implement the core algorithms of Minesweeper, including random mine placement, adjacency calculation, and recursive cell revealing (flood fill).
- **GUI Development:** To create an interactive interface using JavaFX that handles user events (mouse clicks) and updates the visual state of the board in real-time.
- **Event Handling:** To effectively manage user inputs, specifically differentiating between primary (left) and secondary (right) mouse clicks for revealing cells and flagging mines respectively.

# FlowChart:



# System Requirements:

To successfully run and compile this project, the following environment is required:

**Software Requirements:**

- **Java Development Kit (JDK):** Version 17 or later (required for modern JavaFX support).
- **JavaFX SDK:** Libraries for `javafx.controls` and `javafx.fxml`.
- **IDE:** IntelliJ IDEA, Eclipse, or NetBeans.

**Hardware Requirements:**

- **Input:** A mouse or trackpad with distinct left and right-click buttons (essential for flagging mines).
- **Display:** Minimum resolution of 800x600 to display the grid and top bar clearly.

# How to Play:

**Goal:** Reveal all cells on the board that do not contain a mine.

- **Starting a Game:**
  - Launch the application. The board loads automatically with the timer set to 0.
- **Revealing a Square (Left-Click):**
  - Click the Left Mouse Button on a gray square.
  - **Safe Move:** If the square is safe, it will turn light gray. A number may appear indicating how many mines are touching that square (horizontally, vertically, or diagonally).

    - **Game Over:** If you click a square containing a mine, it will explode (turn red), and the game ends immediately.

- **Flagging a Mine (Right-Click):**
    - If you deduce that a square hides a mine, click the Right Mouse Button to place a flag.
    - This visual marker helps you avoid clicking that square accidentally.
    - Right-clicking a flagged square again will remove the flag.
- **Winning:**
    - The game is won automatically when every single safe cell on the grid has been revealed. You do not need to flag all mines to win, but you must open all safe spots.
- **Restarting:**
    - Click the "**Restart**" button at the top of the window at any time to reset the board, timer, and mine locations.

# Program:

## The Board:

The most basic and important element of the game is the Board. The board is implemented in the form of a 2d Array. Board has cells in its rows and columns. These cells contain mines or number of adjacent mines. The board class has the following fields.

The board class has the following functions.

- **placeMines():** Place mines at random indexes in Board.

- countMinesAround(): For every cell, calculate mines in its adjacent cells.
- **isValid():** It checks if the current index being processed is not out of bounds.
- **openZeroCells():** If a clicked cell does not have a mine nor any number , it will open all zero cells in its neighbor. Till hit upon any cell which has a number.
- **revealCell():** Reveals the mines , number of mines in adjacent cells or just an empty cell.
- **checkWin()**: It checks if a player has won the game by subtracting the number of mines from all cells. If the answer is equal to the number of cells revealed that means only mines are unrevealed and the player wins.

## The Cell:

The cell has following properties:

- Is it mine?
- Is it flagged?
- Is it revealed?
- It has a number of mines in adjacent cells.

It has one function.

- **ToggleFlag():** If the cell is not flagged , change it to flagged.

## The GUI:

The GUI of the game is handled in the MinesweeperController class. It has the following structure:

- The basic component of GUI is GridPane , a 2d pane , which contains all cells.
- All cells are buttons.

- All buttons have same event handler, which reveals the cells as following:

  -If it's mine , game over.

  -If it have mines in its adjacent cells , show the number

  -If it has zero mines in its adjacent cells , then open all the zero cells in its neighbor. Till hit upon any cell which has a number.

  -If the user right clicks the mouse , flag it. So players can keep a record of potential mines.

**Procedure:**

1- Put Gridpane on the scene.

2- Wait for the player to click.

3- Handle the click.

4- If the player has lost , show all the mines and change the stage title to “You Lost , Game Over”.

5- If the player has won , change the stage title to “You Won:”.

6- Players can restart the game by clicking the restart button. Which will create a new game , update the board , and reset the timer.

# Code:

## Cell Class:

```
package application;

public class Cell {

   public boolean isFlagged;

   public boolean isMine;

   public boolean isRevealed;

   public int adjacentMines;
```

```
    public Cell() {
        isFlagged = false;
        isMine = false;
        isRevealed = false;
        adjacentMines = 0;
    }

    public void reveal() {
        this.isRevealed = true;
    }

     public void toggleFlag() {

        isFlagged = !isFlagged;
     }

}
```

## Board Class:

```
package application;

public class Board {
    //The Board class will have a 2D array of Cell objects. (grid)
      //It's responsible for creating them, placing mines and then
calculating the adjacent mine numbers for each cell.

    private int rows;
    private int cols;
    private int numMines;

    public Cell[][] grid; //!

    //constructor!!!
    public Board(int rows, int cols, int numberOfMines) {
        this.rows = rows;
        this.cols = cols;

        this.numMines = numberOfMines;
        grid = new Cell[rows][cols];
```

```
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                grid[i][j] = new Cell();
            }
        }

        placeMines(numberOfMines);
        calculateAdjacentMines();

    }

//     public void printBoard() {
//
//         for (int i = 0; i < rows; i++) {
//             for (int j = 0; j < cols; j++) {
//
//                 if (grid[i][j].isFlagged) {
//                     System.out.print(" F ");
//                 } else if (grid[i][j].isRevealed == false) {
//                     System.out.print(" . ");
//                 } else if (grid[i][j].isMine) {
//                     System.out.print(" X ");
//                 } else {
//                                             System.out.print("  "  +
grid[i][j].adjacentMines + " ");
//                 }
//             }
//             System.out.println();
//         }
//     }

    private void placeMines(int totalMines) {
        int numberOfMinesPlaced = 0;

          //int randomNumber = (int) (Math.random() * max); // (0 to
max-1)
        while (numberOfMinesPlaced < totalMines) {
            int rRowIndex = (int) (Math.random() * rows);
            int rColIndex = (int) (Math.random() * cols);

            //check!!!
```

```
            if (grid[rRowIndex][rColIndex].isMine ≠ true) {

                grid[rRowIndex][rColIndex].isMine = true;

                numberOfMinesPlaced++;
            }
        }
    }
    //-------------calculate int adjacentMines-------------

    //check if the cell is valid (IndexOutOfBoundsException)
    private boolean isValid(int r, int c) {
        if (r < rows && r ⩾ 0 && c < cols && c ⩾ 0) {
            return true;
        } else {
            return false;
        }
    }

     //look at all the 8 neighbours of the cell and return the mines
found
    private int countMinesAround(int r, int c) {
        int count = 0;

          if (isValid(r - 1, c - 1) && grid[r - 1][c - 1].isMine ==
true) {
            count++;
        }
        if (isValid(r - 1, c) && grid[r - 1][c].isMine == true) {
            count++;
        }
          if (isValid(r - 1, c + 1) && grid[r - 1][c + 1].isMine ==
true) {
            count++;
        }
        if (isValid(r, c - 1) && grid[r][c - 1].isMine == true) {
            count++;
        }
        //?
        if (isValid(r, c + 1) && grid[r][c + 1].isMine == true) {
            count++;
```

```
        }
          if (isValid(r + 1, c - 1) && grid[r + 1][c - 1].isMine ==
true) {
              count++;
        }
        if (isValid(r + 1, c) && grid[r + 1][c].isMine == true) {
              count++;
        }
          if (isValid(r + 1, c + 1) && grid[r + 1][c + 1].isMine ==
true) {
              count++;
        }
        return count;
    }

    //final step
    private void calculateAdjacentMines() {

        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                grid[i][j].adjacentMines = countMinesAround(i, j);
            }
        }
    }
    //.....................................//

    private void openZeroCells(int r, int c) {

        if (!isValid(r, c)) {
            return;
        }
        if (grid[r][c].isRevealed) {
            return;
        }
        if (grid[r][c].isFlagged) {
            return;
        }

        grid[r][c].reveal();

        //recursion
```

```
        if (grid[r][c].adjacentMines == 0) {
            openZeroCells(r - 1, c - 1);
            openZeroCells(r - 1, c);
            openZeroCells(r - 1, c + 1);
            openZeroCells(r, c - 1);
            openZeroCells(r, c + 1);
            openZeroCells(r + 1, c - 1);
            openZeroCells(r + 1, c);
            openZeroCells(r + 1, c + 1);
        }

    }

    //get the cell to reveal
    public Cell revealCell(int r, int c) {
        if (!isValid(r, c)) {
            return null;
        }
        if (grid[r][c].isMine) {

            grid[r][c].reveal();
              return grid[r][c]; //it would return and then game over
is going to handle it.
        }

        openZeroCells(r, c);

        return grid[r][c];
    }



    public Cell getCell(int r, int c) {
        if (isValid(r, c)) {
            return grid[r][c];
        }
        return null;
    }

    //if a player win
     public boolean checkWin() {
```

```
        int revealCount = 0;
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                if (grid[i][j].isRevealed) {
                    revealCount++;
                }
            }
        }
        if (revealCount == (rows * cols) - numMines) {
            return true;
        } else {
            return false;
        }
    }
}
```

## GameState Class:

```
package application;

public enum GameState {
    PLAYING,
    WON,
    LOST
}
```

## Main Class:

```
package application;

import javafx.application.Application;
import javafx.fxml.FXMLLoader;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.scene.image.Image;
import javafx.stage.Stage;

import java.util.Objects;
```

```
public class MinesweeperApp extends Application {

   @Override
   public void start(Stage stage) throws Exception {
       Parent root =
FXMLLoader.load(Objects.requireNonNull(getClass().getResource("/Mine
sweeper.fxml")));

       stage.setTitle("Minesweeper");
       Image icon = new
Image(Objects.requireNonNull(getClass().getResourceAsStream("/pngegg
.png")));
       stage.getIcons().add(icon);
       stage.setScene(new Scene(root));
       stage.show();
   }

   public static void main(String[] args) {
       launch(args);
   }
}
```

## Main Controller:

```
package application;

import javafx.fxml.FXML;

import javafx.scene.control.Label;
import javafx.scene.input.MouseButton;
import javafx.scene.input.MouseEvent;
import javafx.scene.layout.GridPane;
import javafx.scene.paint.Color;

public class MinesweeperController {

   @FXML
   private GridPane gridPane;
   @FXML
   private Label statusLabel;
```

```
    private Board board;

    private final int ROWS = 20;
    private final int COLS = 20;
    private final int MINES = 80;

    private boolean gameOver = false;

    private int flagsPlaced = 0;

    public void initialize() {
        startNewGame();
    }

    @FXML
    void resetGame() {
        startNewGame();
    }

    private void startNewGame() {
        board = new Board(ROWS, COLS, MINES);
        gameOver = false;


        flagsPlaced = 0;
        updateStatusLabel();

        gridPane.getChildren().clear();

        for (int r = 0; r < ROWS; r++) {
            for (int c = 0; c < COLS; c++) {
                Cell cell = board.getCell(r, c);
                CellButton button = new CellButton(r, c, cell);
                button.setOnMouseClicked(e ->
handleMouseClick(button, e));
                gridPane.add(button, c, r);
            }
        }
    }
```

```
private void updateStatusLabel() {
    if (gameOver) return;

    int remaining = MINES - flagsPlaced;
    statusLabel.setText("Mines: " + remaining);


}

private void handleMouseClick(CellButton btn, MouseEvent e) {
    if (gameOver) return;


    if (e.getButton() == MouseButton.SECONDARY) {


        if (!btn.getCell().isRevealed) {


            if (btn.getCell().isFlagged) {

                flagsPlaced--;
            } else {

                flagsPlaced++;
            }

            btn.getCell().toggleFlag();
            updateButton(btn);


            updateStatusLabel();
        }
    }

    else if (e.getButton() == MouseButton.PRIMARY) {
        if (btn.getCell().isFlagged) return;

        board.revealCell(btn.getRow(), btn.getCol());
```

```
            if (btn.getCell().isMine) {
                gameOver = true;
                statusLabel.setText("GAME OVER!");
                statusLabel.setTextFill(Color.RED);
                revealAll();
            } else if (board.checkWin()) {
                gameOver = true;
                statusLabel.setText("YOU WIN!");
                statusLabel.setTextFill(Color.GREEN);
                revealAll();
            } else {
                updateBoardUI();
            }
        }
    }

    private void updateBoardUI() {
        for (javafx.scene.Node node : gridPane.getChildren()) {
            if (node instanceof CellButton) {
                updateButton((CellButton) node);
            }
        }
    }

    private void updateButton(CellButton btn) {
        Cell cell = btn.getCell();

        if (cell.isRevealed) {
            btn.setDisable(true);
            if (cell.isMine) {
                btn.setText("💣");
                btn.setStyle("-fx-background-color: #ff5555;
-fx-opacity: 1;");
            } else {
                if (cell.adjacentMines > 0) {
                    btn.setText(String.valueOf(cell.adjacentMines));

                    switch(cell.adjacentMines) {
                        case 1: btn.setTextFill(Color.BLUE); break;
                        case 2: btn.setTextFill(Color.GREEN); break;
                        case 3: btn.setTextFill(Color.RED); break;
```

```
                        case 4: btn.setTextFill(Color.DARKBLUE);
break;
                        default: btn.setTextFill(Color.BLACK);
                    }
                } else {
                    btn.setText("");
                }
                btn.setStyle("-fx-background-color: #dddddd;
-fx-opacity: 1;");
            }
        } else {

            btn.setDisable(false);
            if (cell.isFlagged) {
                btn.setText("🏳");
                btn.setTextFill(Color.BLACK);
                btn.setStyle("-fx-background-color: #ffff99;");
            } else {
                btn.setText("");
                btn.setStyle(null);
            }
        }
    }

    private void revealAll() {
        for (int r=0; r<ROWS; r++) {
            for (int c=0; c<COLS; c++) {
                board.getCell(r, c).isRevealed = true;
            }
        }
        updateBoardUI();
    }
}
```

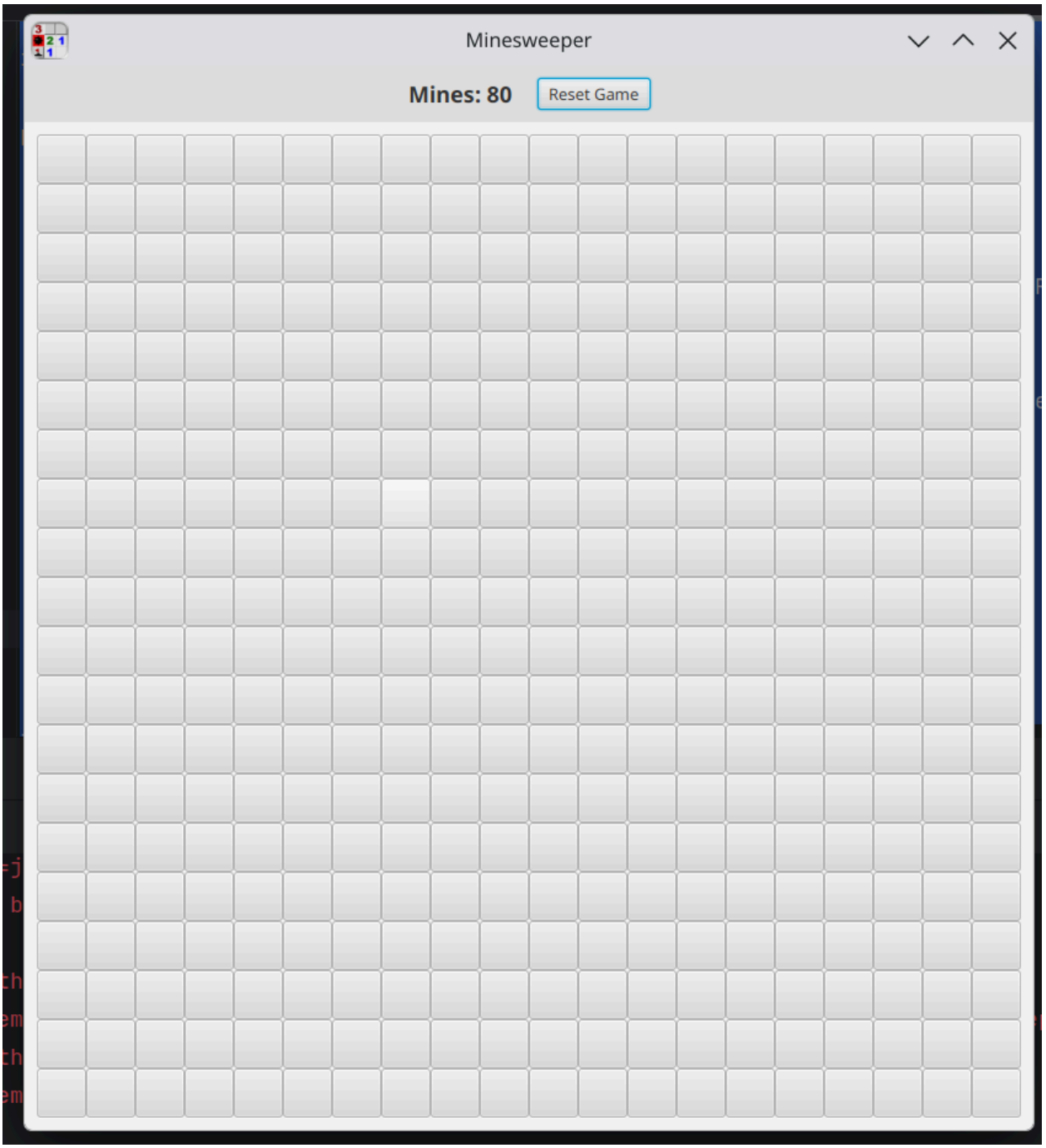
## FXML:

```
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.scene.control.Button?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.layout.BorderPane?>
<?import javafx.scene.layout.GridPane?>
```

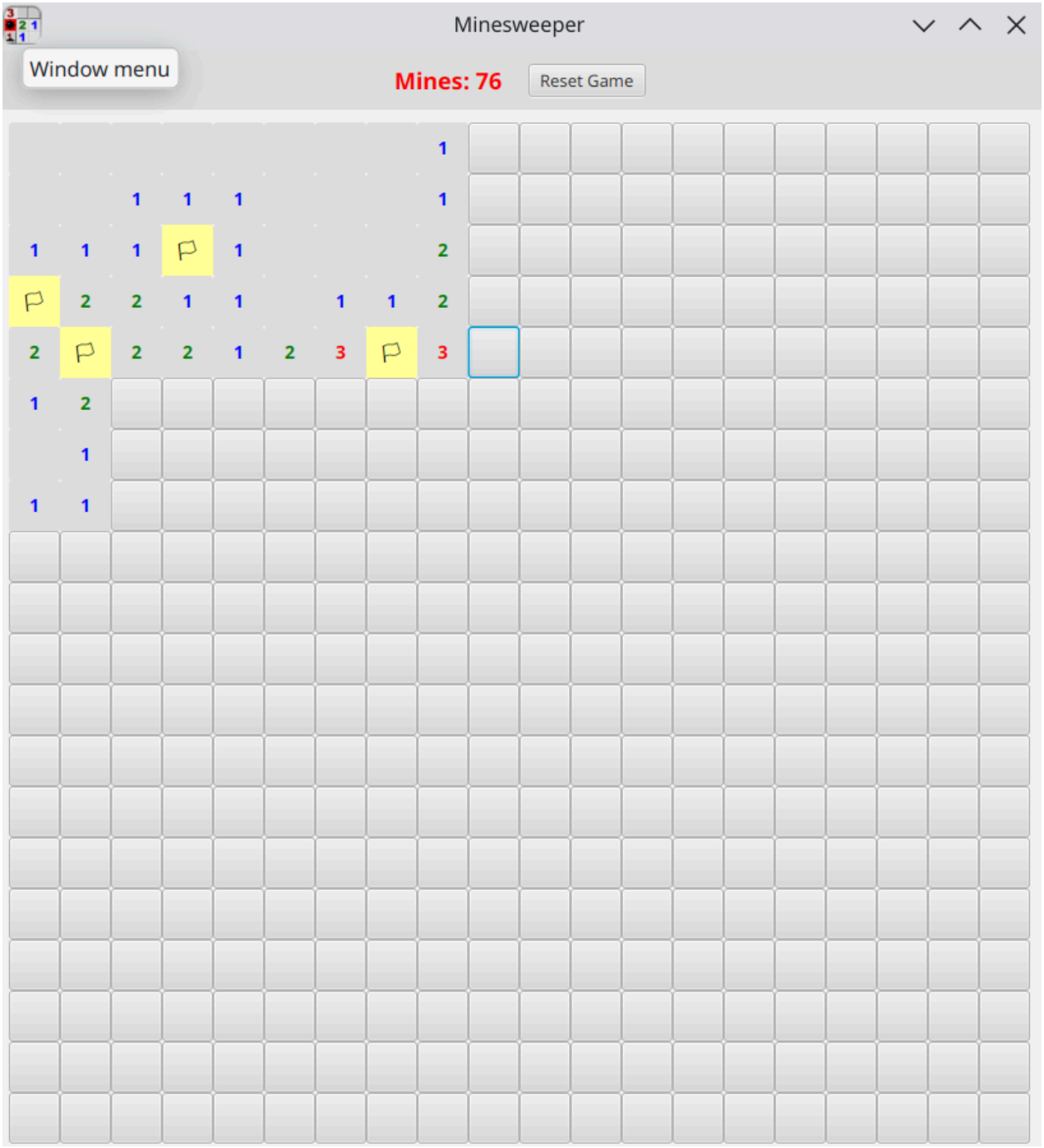
```
<?import javafx.scene.layout.HBox?>
<?import javafx.scene.text.Font?>

<BorderPane maxHeight="-Infinity" maxWidth="-Infinity"
minHeight="-Infinity" minWidth="-Infinity"
           xmlns="http://javafx.com/javafx/17"
xmlns:fx="http://javafx.com/fxml/1"
           fx:controller="application.MinesweeperController">

   <top>
       <HBox alignment="CENTER" spacing="20.0" style="-fx-padding:
10; -fx-background-color: #ddd;" BorderPane.alignment="CENTER">
           <children>
               <Label fx:id="statusLabel" text="Minesweeper"
textFill="#333">
                   <font>
                       <Font name="System Bold" size="18.0" />
                   </font>
               </Label>
               <Button mnemonicParsing="false" onAction="#resetGame"
text="Reset Game" />
           </children>
       </HBox>
   </top>

   <center>
       <GridPane fx:id="gridPane" alignment="CENTER"
style="-fx-padding: 10;" BorderPane.alignment="CENTER">
       </GridPane>
   </center>

</BorderPane>
```

# Output:

## Game Start:



GamePlay:



Game Over: